Medical Image Analysis 99 (2025) 103367

Contents lists available at ScienceDirect

Medical Image Analysis

journal homepage: www.elsevier.com/locate/media

Efficient anatomical labeling of pulmonary tree structures via deep point-graph representation-based implicit fields☆

Kangxian Xie [a,b], Jiancheng Yang [a,*], Donglai Wei [b], Ziqiao Weng [c], Pascal Fua [a]

[a] Computer Vision Laboratory, Swiss Federal Institute of Technology Lausanne (EPFL), Lausanne 1015, Switzerland
[b] Boston College, Chestnut Hill, MA 02467, USA
[c] University of Sydney, Camperdown NSW 2050, Australia

ARTICLE INFO

Dataset link: https://github.com/M3DV/pulmonary-tree-labeling

MSC:
68T45
62P10
68U10
68U05
05C90

Keywords:
Pulmonary tree labeling
Graph
Point cloud
Implicit function
3D deep learning

ABSTRACT

Pulmonary diseases rank prominently among the principal causes of death worldwide. Curing them will require, among other things, a better understanding of the complex 3D tree-shaped structures within the pulmonary system, such as airways, arteries, and veins. Traditional approaches using high-resolution image stacks and standard CNNs on dense voxel grids face challenges in computational efficiency, limited resolution, local context, and inadequate preservation of shape topology. Our method addresses these issues by shifting from dense voxel to sparse point representation, offering better memory efficiency and global context utilization. However, the inherent sparsity in point representation can lead to a loss of crucial connectivity in tree-shaped structures. To mitigate this, we introduce graph learning on skeletonized structures, incorporating differentiable feature fusion for improved topology and long-distance context capture. Furthermore, we employ an implicit function for efficient conversion of sparse representations into dense reconstructions end-to-end. The proposed method not only delivers state-of-the-art performance in labeling accuracy, both overall and at key locations, but also enables efficient inference and the generation of closed surface shapes. Addressing data scarcity in this field, we have also curated a comprehensive dataset to validate our approach. Data and code are available at https://github.com/M3DV/pulmonary-tree-labeling.

1. Introduction

In recent years, since pulmonary diseases (Decramer et al., 2008; Marcus et al., 2000; Nunes et al., 2017) have become the leading causes of global mortality (Quaderi and Hurst, 2018), pulmonary research has gained increasing attention. In studies related to pulmonary disease, understanding pulmonary anatomies through medical imaging is important due to the known association between pulmonary disease and inferred metrics from lung CT images (Charbonnier et al., 2019; Kirby et al., 2019; Qin et al., 2019; Shaw et al., 2002; Loud et al., 2001; Shen et al., 2014).

The complex tree-shaped pulmonary structures — airways, arteries, and veins, as depicted by Fig. 1 — have high branching factors and play a crucial role in the respiratory system in the lung. Multi-class semantic segmentation of the pulmonary trees in volumetric images, where each class represents a specific division or branch of the tree according to the medical definition of the pulmonary segments, is an effective approach to modeling their intricacies, with the challenge primarily lying in the bifurcation locations and distal branches (Yu et al., 2023). In pulmonary tree labeling, the derived quantitative characteristics (Kirby et al., 2019; Smith et al., 2014; Shaw et al., 2002) not only associate with lung diseases and pulmonary-related medical applications (Loud et al., 2001; Shen et al., 2014) but are also crucial for surgical navigation (Qin et al., 2019). This work focuses on investigating and developing deep-learning methods for efficient and accurate anatomical labeling of pulmonary airways, arteries, and veins.

Among deep learning approaches, convolutional neural networks (CNN) have become the *de facto* standard approach to semantic segmentation (Zhou et al., 2018, 2019). One of their strengths is that they yield volumes with well-defined surfaces. However, they are computationally demanding when processing large 3D volumes and often deliver unsatisfactory results when operating at a reduced resolution (Fig. 2(a)) or on local patches (Fig. 2(b)), either leading to a lack of details or global context. In contrast, point-cloud is a sparse representation of 3D shapes without usable surface, but it comes with lower computational requirements while preserving global structures (Fig. 2(c)). Besides, due to the inherent tree-shaped targets, graph modeling (Fig. 2(d)) that

☆ This work was conducted during K. Xie's research internship at EPFL and Boston College.

* Corresponding author.
E-mail address: jiancheng.yang@epfl.ch (J. Yang).

https://doi.org/10.1016/j.media.2024.103367
Received 4 December 2023; Received in revised form 6 June 2024; Accepted 2 October 2024
Available online 8 October 2024
1361-8415/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

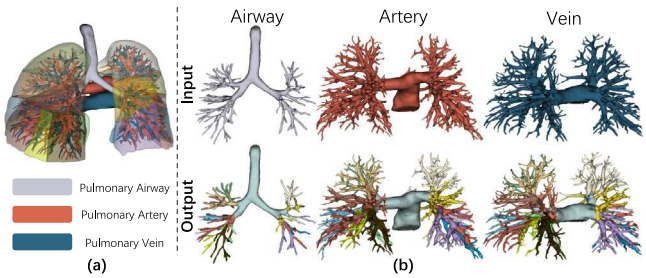

Fig. 1. Pulmonary Tree Labeling. (a) The pulmonary tree consists of three anatomic structures (airway, artery and vein). (b) Given a binary volume representing a tree structure as input, we label each voxel into one of 19 classes (18 pulmonary segments + 1 background out of lung) based on branching regions.

preserves the topology and emphasizes learning on key locations is also visible (Yu et al., 2023; Tan et al., 2021; Xie et al., 2022). Nevertheless, extracting usable surfaces from point clouds or graphs is non-trivial.

Considering the challenges mentioned above, as illustrated in Fig. 2(e), we would like to develop an optimal automated solution to this task, which is computationally efficient, and reconstruct the 3D model with continuous surface definition and topology preservation, specifically focusing on the key structural points. To this end, we introduce a novel approach that enables deep infusion of skeleton-graph context into the point-based representations, with an implicit method to yield a feature field, which allows for efficient and denoised dense volume reconstruction. The proposed *Implicit Point-Graph Network (IPGN)* includes point and graph encoders, combined with multiple *Point-Graph Fusion* layers as *Point-Graph Network (PGN)* for deep feature fusion and an *Implicit Point Module* to model the implicit surface in 3D, allowing for fast classification inference at arbitrary locations. Thanks to the flexibility of implicit representations, the IPGN trained for pulmonary tree labeling can be further extended to pulmonary segment reconstruction by simply modifying the inference method (Section 6).

As illustrated by Fig. 9, our approach produces high-quality dense reconstructions of pulmonary structures at an acceptable computational cost. To evaluate it quantitatively, we compiled the Pulmonary Tree Labeling (PTL) dataset illustrated in Fig. 1. It contains manual annotations for 19 different components of pulmonary airways, arteries, and vein, which will be made publicly available as a multi-class semantic segmentation benchmark for pulmonary trees. Our method achieves state-of-the-art performance on this dataset while being the most computationally efficient. Our contributions can be summarized as:

- We present a new method for analyzing 3D pulmonary tree structures, transitioning from dense voxel to sparse representation for improved memory efficiency and global context understanding.
- To enhance topology preservation and connectivity, we propose differentiable point-graph feature fusion on sparse points and skeletonized tree structures.
- It further incorporates an implicit function to efficiently convert sparse representations into dense reconstructions.
- We have compiled a comprehensive Pulmonary Tree Labeling (PTL) dataset, achieving superior labeling accuracy over previous methods at both the voxel (overall performance) and graph (key-location performance) levels, while also demonstrating significantly higher efficiency.

2. Related works

2.1. Pulmonary anatomy segmentation

In pulmonary-related studies, image-based understanding of pulmonary anatomies is important as metrics inferred from lung imaging have shown to be related to severity, development and therapeutic outcome of pulmonary diseases (Charbonnier et al., 2019; Kirby et al., 2019; Qin et al., 2019; Shaw et al., 2002; Loud et al., 2001; Shen et al., 2014).

Previously, CNN-based methods have been tailored for comprehension of different pulmonary anatomies, such as pulmonary lobe (Gerard and Reinhardt, 2019), airway (Zhang et al., 2023). artery-vein (Nardelli et al., 2018; Qin et al., 2020) and fissure (Gerard et al., 2019). Among various pulmonary anatomies, tree-shaped pulmonary structures have drawn a lot of research attention. For pulmonary airway tree, not only is proper segmentation crucial for surgical navigation (Qin et al., 2019), segmentation-derived attributes like airway counts (Kirby et al., 2019), thickness of wall (Smith et al., 2014) and morphological changes (Shaw et al., 2002) also have association with lung diseases. For pulmonary vasculature including arteries and veins, quantitative attributes extracted from segmentation are also commonly applied in multiple pulmonary-related medical applications like emboli detection (Loud et al., 2001) and hypertension (Shen et al., 2014).

Previous works specifically on pulmonary tree segmentation either apply graph-only modeling or leverage graph-CNN multi-task learning. A Tree-Neural Network (Yu et al., 2023) leverages handcrafted features on a tailored hyper-graph for pulmonary bronchial tree segmentation to address the inherent problem of overlapping distribution of features. A recent work (Xie et al., 2022) on airway segmentation proposes graph modeling to incorporate structural context to local CNN feature from each airway branch. Yet, both works merely provide labeling to pulmonary tree branches that are pre-defined at pixel level, and thus are not for semantic segmentation, where defining accurate borders between neighboring branches remains a challenge. Applicable to binary or raw images of pulmonary structures, SG-Net (Tan et al., 2021) employs CNN features for detecting of landmarks, constructing graphs for CNN-graph multi-task learning.

Although these methods are graph-based, the graph construction procedure varies. While one treats each pre-defined branch as a node (Xie et al., 2022), disrupting the original spatial structure, another is parameter-based (Yu et al., 2023), causing the quality of the constructed tree highly dependent on the parameter selection, and finally, the SG-Net (Tan et al., 2021) established its graph node by learned landmark prediction, whose structural quality cannot be ensured. In our setup, the skeleton graphs are based on a thinning algorithm (Lee et al., 1994), with no modeling or hyper-parameters tuning involved, and all spatial and connection relationships between tree segments are acquired directly from the original dense volume (Fig. 1). Additionally, as CNN methods incur memory footprint that grows in cubical, they are expensive when facing large 3D volumes.

2.2. 3D deep learning

Deep learning on 3D geometric data is a specialized field that focuses on adapting neural networks to manage non-Euclidean domains, such as point clouds, graphs (or meshes), and implicit surfaces. 3D CNNs for voxel representations can also handle shapes (Wu et al., 2015), but encounter limitations due to their dense nature, leading to high memory and computational demands. As a result, these representations often necessitate downsampling for modeling at lower resolutions (Fig. 2(a)), or the use of sliding-window techniques (Fig. 2(b)), which unfortunately restrict them to capturing only local context, thereby failing to effectively utilize global context.

Point-based methods have emerged as a novel approach for 3D geometric modeling. As sparse 3D data (Fig. 2(c)), point cloud can be modeled in a variety of ways, such as multi-layer perception (Qi et al., 2016, 2017), convolution (Li et al., 2018; Liu et al., 2019), graph (Lin et al., 2020) and transformer (Yang et al., 2019; Zhao et al., 2021). Point-based methods have also been validated in the medical image analysis domain (Yang et al., 2021; Jin et al., 2023). However, point

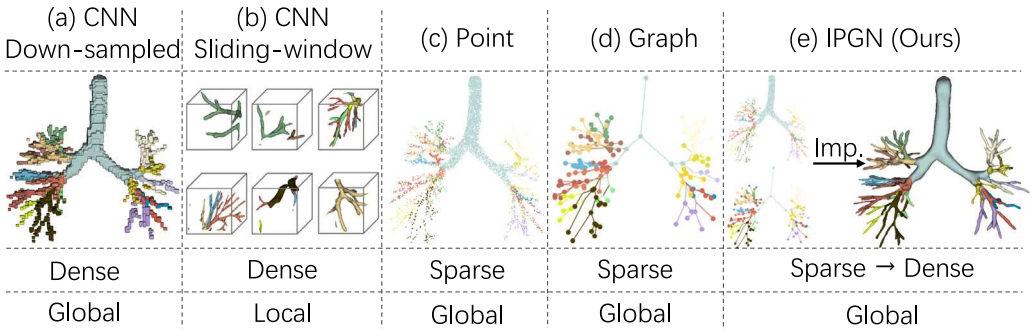

Fig. 2. A Comparison of Data Representation for Pulmonary Tree Labeling. The CNN-based voxel methods are either low-resolution (down-sampled) or local (sliding-window). The standard sparse representation like point and graph is global but it is not trivial to reconstruct high-quality dense volume. Our method that combines point, graph, and implicit fields produces high-quality dense reconstruction efficiently.

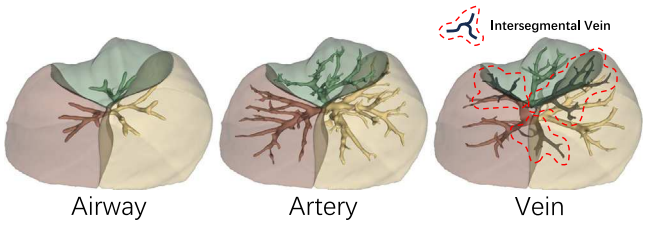

Fig. 3. Anatomy of Pulmonary Trees and Pulmonary Segments. Each pulmonary tree branch corresponds to a pulmonary segment. The intersegmental vein is highlighted in red, which lies along the pulmonary segment border.

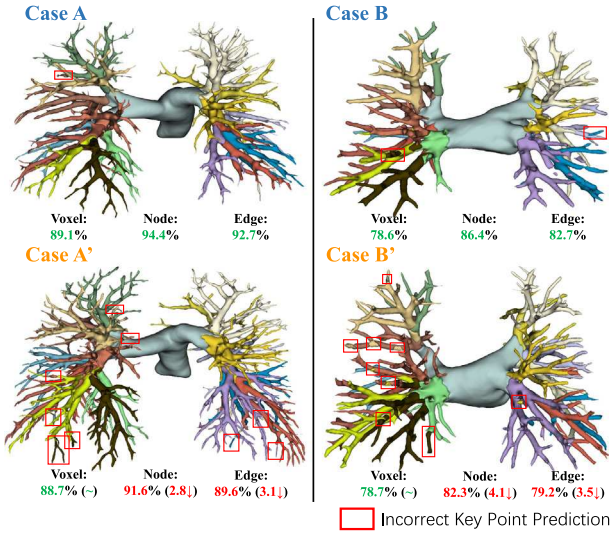

Fig. 4. Sample Illustration with Voxel-level and Graph-level Metrics. Due to the large number of voxels, voxel-level metric often overlooks labeling at key regions. Given two test samples with similar voxel performance, the sample with better graph-level performance (Case A and B) tends to perform better at key points than its counterpart (Case A' and B').

cloud, as sparse representation of 3D shape, does not provide a closed surface.

Since the initial introduction (Kipf and Welling, 2016), graph learning has become a powerful tool for analyzing graph-structured data (Fig. 2(d)), being widely applied to medical images analysis (Yu et al., 2023; Xie et al., 2022; Tan et al., 2021) or bioinformatics (Jumper et al., 2021). Meshes represent a specialized form of graph, typically characterized by a collection of triangular faces to depict the object surfaces, and are extensively employed in the field of graphics. Additionally, they also maintain some applications in the domain of medical imaging (Wickramasinghe et al., 2020, 2022). While point-based methods enable efficient sparse-representation learning on dense volumetric data, graph learning is lightweight and learns structural context within geometric data. Combining point and graph methods is advantageous in our task. However, extracting surfaces (or dense volumes) from sparse point-based or graph-based prediction is non-trivial, which leads us to introduce implicit surface representation to address this issue (Fig. 2(e)).

Deep implicit fields/functions have been successful at modeling 3D shapes (Chen and Zhang, 2018; Chibane et al., 2020; Huang et al., 2022; Mescheder et al., 2018; Park et al., 2019; Peng et al., 2020; Yang et al., 2022). Typically, the implicit function predicts occupancy or signed distance at continuous locations, thus capable of reconstructing 3D shapes at arbitrary resolutions (Kuang et al., 2022). Moreover, implicit networks could be trained with randomly sampled points, which lowers the computation burden during training. These characteristics suggest that implicit fields have inherent advantages when reconstructing the sparse prediction of pulmonary tree labeling into dense.

Additionally, when evaluating the segmentation performance, we consider the presence of intersegmental veins and the potential ambiguity in their class assignment. Intersegmental veins are veins that lie along the border between two neighboring pulmonary segments (Oizumi et al., 2014), highlighted in Fig. 3. As pulmonary tree branches are involved in the boundary definition of pulmonary segments (Kuang et al., 2022), intersegmental veins pose an inherent challenge in their class definition. To address this issue, we mask out the intersegmental veins during the evaluation and only focus on segmentation of the pulmonary airway, artery, and veins within each individual segment.

3. Problem formulation

3.1. Pulmonary tree labeling

In this study, we address the pulmonary tree labeling problem in 3D CT images. Specifically, given a binary volumetric image of a pulmonary tree, our objective is to provide an accurate 19-class (18 foreground classes + 1 background) segmentation of the pulmonary airway, artery, and vein trees into segmental-level branches and extra-pulmonary components, demonstrated in Fig. 1 and Fig. 3. Each foreground pixel will be assigned to its respective semantic class.

While this task reconstructs a high-resolution 3D shape with dense annotations, the primary challenge lies in the correct labeling of key branching points, and distal end points (Yu et al., 2023). As the pixels contained in the 3D image have large quantities, learning signals targeting these key locations in the tree structures is overpowered. In Fig. 4, a few test samples illustrate that overall voxel-level dice performances do not translate to performances at key locations due to the large volume of pixels that could be easily classified. Therefore, another key

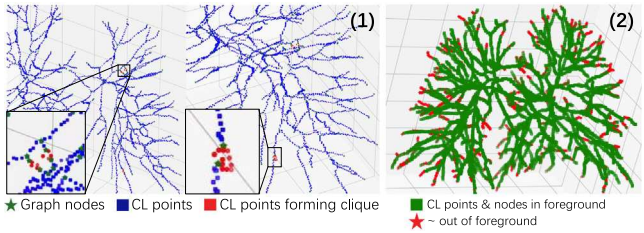

Fig. 5. Skeleton Graphs Directly from Bumgarner and Nelson (2022) **and Two Types of Imperfections.** (1) The centerline (CL) points, which represent the path of the tree branch as discrete and connected coordinates, might contain cliques. (2) The coordinates of leaf graph nodes might fall outside the foreground volume.

component of this labeling problem is to ensure a particularly precise annotation at these important locations. To this end, we preprocess the volume and derive an approximate representation of key points as nodes in a skeleton graph (Section 3.2.2) and perform graph node/edge classification for evaluation.

3.2. Dataset

3.2.1. Overview

From multiple medical centers, we compile the Pulmonary Tree Labeling (PTL) dataset containing annotated pulmonary tree structures for 800 subjects. For each subject, there are 3 volumetric images containing its pulmonary airway, artery, and vein, illustrated in Fig. 1(b). In each volume, the annotations consist of 19 classes, where labels 1 to 10 represent branches located in the left lung, labels 11 to 18 represent those in the right lung, and class 19 represents extra-pulmonary structures.

All 3D volumes have shapes $N \times 512 \times 512$, where 512×512 represents the CT slice dimension, and N denotes the number of CT slices, ranging from 181 to 798. Z-direction spacings of these scans range from 0.5 mm to 1.5 mm. Manual annotations are produced by a junior radiologist and verified by a senior radiologist. During the modeling process, the images are converted to binary as input, and the original annotated image is the prediction target. The dataset is randomly split into 70% train, 10% validation, and 20% test samples.

3.2.2. Skeleton graph

As discussed in Section 3.1, other than the voxel-level labeling on the entire image foreground, labeling of key branching points is considered a crucial component of the task. As the branching key points and distal endpoints are easily overwhelmed by image pixels in quantity, we construct a graph for each 3D volume, with nodes as approximate representations of the key points for evaluation (Fig. 2(d)).

The skeleton graph dataset is derived from the PTL dataset. Given a 3D volume containing a medical shape, such as a tree-shaped pulmonary structure, we utilize a software application vesselvio (Bumgarner and Nelson, 2022), which applies a well-known thinning algorithm (Lee et al., 1994), to derive the shape's centerlines, which are used to construct a connectivity-preserving skeleton tree graph in 3D space, illustrated by Fig. 2(d). In the resulting tree graphs, centerlines become edges that represent anatomical connections, centerline intersections and distal endpoints become nodes that represent key points in the pulmonary structures that require specific attention. In this manually derived graph dataset, we specialized a labeling procedure to ensure annotation consistency between the software-generated graph and the original volume. In this procedure, we start by labeling edges before node so that the deterministic edge label could improve the quality of node labels that are non-deterministic at class intersections. More specifically, the annotation process can be summarized as the following:

Graph simplification. From software (Bumgarner and Nelson, 2022) output, for each edge e, inter-connected point coordinates sparsely representing the centerline of e are also recorded (Fig. 5 CL points) and they are key to the subsequent label assignment. However, due to software inconsistency, cliques occur as displayed in Fig. 5 (1). To best prepare for the subsequent labeling, we remove the cliques by only keeping the shortest path between two nodes.

Edge labeling. For any $e \in E$, we recover the labels of the points on the centerline from the original volume. Then, majority voting is applied on the centerline labels to determine the final label for e.

Node labeling. If a node is a leaf node, the node inherits the label of that edge. This has advantage over seeking for label directly from the volume because due to software inconsistency, the coordinates of leaf nodes might not fall into volume foreground, illustrated in Fig. 5 (2). For nodes with multiple edges, majority voting is applied on the labels of all connected edges to determine the node's label.

3.3. Evaluation metrics

This procedure enables a robust ground truth annotation of the graph, reflecting both the local and global structural information contained within the original 3D volume. Therefore, the task within this graph dataset involves performing a 19-class classification for both graph nodes and edges.

For performance evaluation, micro-averaged dice scores are used as metrics due to class imbalance. For voxel-level results on dense volume, we employ the dice score to assess the overlap between the predicted segmental-level branch and the ground truth branch, providing a measure of similarity. As illustrated in Fig. 4, voxel-level results overlook performances at some critical locations, in which volumes are significantly less. To bridge this discrepancy, we also evaluate the performance at the skeleton graph-level (Section 3.2.2), to estimate performance at key points of the tree structure. For graph-level node and edge classification, we also utilize dice as metrics. The dice score can be used to assess the similarity of the predicted graph structure with the ground truth graph structure.

4. Methodology

Our objective is centered around the anatomical labeling of pulmonary trees. That is to say, given an input of binary volumes of a pulmonary tree—derivable from manual annotations or model predictions—we aim to execute a 19-class semantic segmentation on every non-zero voxel. However, diverging from standard methodologies reliant on CNNs, we down-sample the raw dense volume to a point cloud while concurrently extracting a skeleton graph (Section 3.2.2). Our approach (Section 4.1) engages in representation learning individually on the sparse representations of both the point cloud and the skeleton graph, and a fusion module is employed to perform deep integration of point-graph representations (Section 4.2). Ultimately, to reconstruct the predictions based on sparse representations back to dense, we introduce implicit fields to facilitate efficient reconstruction (Section 4.3). To improve the readability of methodology, we describe the notations in Table 1.

4.1. Implicit point-graph network architecture

Given a binary volumetric image of a pulmonary tree (Fig. 6(a)), a graph (Fig. 6(b)) is constructed with VesselVio (Bumgarner and Nelson, 2022) from the original volume following the procedure in 3.2.2, and a set of points ($6k$) are randomly sampled from the foreground voxels to construct a point cloud (Fig. 6(c), referred to as original point cloud). While the original point cloud is a sparse and downsampled version of the foreground volume, the graph represents a skeleton of the pulmonary tree.

Table 1
Notations and corresponding descriptions. This table presents the notations used to represent point and graph elements within the architecture. The notations are organized into two stages: Point-Graph Feature Fusion and Implicit Dense Volume Reconstruction.

Notation	Definition
Point-Graph Feature Fusion	
$\mathbf{P} = \{\mathbf{p}_1, \dots, \mathbf{p}_m\} \in \mathbb{R}^{M\times 3}$	The original point cloud with M points.
$\mathbf{p} = \{x, y, z\} \in \mathbb{R}^3$	A single point in the original point cloud $\mathbf{P}$, $\mathbf{p} \in \mathbf{P}$.
$\mathbf{p}^{(i)} = \{\mathbf{p}_1^{(i)}, \dots, \mathbf{p}_m^{(i)}\} \in \mathbb{R}^{M\times 128}$	Features of the original point cloud P after the ith layer.
$\mathbf{p}^{(i)} \in \mathbb{R}^{128}$	Feature of a single point $\mathbf{p}$ after the ith layer, $\mathbf{p}^{(i)} \in \mathbf{P}^{(i)}$.
$\mathbf{G} = \{\mathbf{g}_1, \dots, \mathbf{g}_n\} \in \mathbb{R}^{N\times 3}$	All N nodes in a skeleton graph.
$\mathbf{g} \in \mathbb{R}^3$	A single node in the graph node set $\mathbf{G}$, $\mathbf{g} \in \mathbf{G}$.
$\mathbf{G}^{(i)} = \{\mathbf{g}_1^{(i)}, \dots, \mathbf{g}_n^{(i)}\} \in \mathbb{R}^{N\times 128}$	Features of the graph node set $\mathbf{G}$ after the ith layer.
$\mathbf{g}^{(i)} \in \mathbb{R}^{128}$	Feature of a single node $\mathbf{g}$ after the ith layer, $\mathbf{g}^{(i)} \in \mathbf{G}^{(i)}$.
$\mathcal{F}_1 : \mathbb{R}^{128} \to \mathbb{R}^{128}$	An MLP for ball queried points for graph branch.
$\mathcal{F}_2 : \mathbb{R}^{256} \to \mathbb{R}^{128}$	An MLP for graph feature propagation in point branch.
$\mathcal{G} : \mathbb{R}^{N\times 128} \to \mathbb{R}^{N\times 128}$	A GNN to update graph node feature after $\mathcal{F}_1$.
Implicit Dense Volume Reconstruction	
$\mathbf{p}_q = \{x_q, y_q, z_q\}$	An arbitrary query point, $\mathbf{p}_q \in$ or $\notin \mathbf{P}$.
$\mathbf{p}_k \in \mathbb{R}^3$	$\mathbf{p}_q$'s kth nearest neighbor point in the original point cloud $\mathbf{P}$, $\mathbf{p}_k \in \mathbf{P}$.
$\mathbf{p}_k^{(i)} \in \mathbb{R}^{128}$	$\mathbf{p}_k$'s feature from ith layer point features $\mathbf{P}^{(i)}$.
$\mathbf{z}_k = \{\mathbf{p}_k^{(0)} \frown \mathbf{p}_k^{(1)} \frown \dots \frown \mathbf{p}_k^{(i)}\} \in \mathbb{R}^{128\times(i+1)}$	$\mathbf{p}_k$'s multi-stage feature by concatenating multi-stage point features.
$\mathbf{z}_q \in \mathbb{R}^{128\times(i+1)}$	$\mathbf{p}_q$'s multi-stage feature using feature propagation from $\{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_k\}$.
$\mathcal{H} : \mathbb{R}^{128\times(i+1)} \to \mathbb{R}^{19}$	An MLP to project the multi-stage features $\mathbf{z}_q$ of a query point $\mathbf{p}_q$ to prediction.

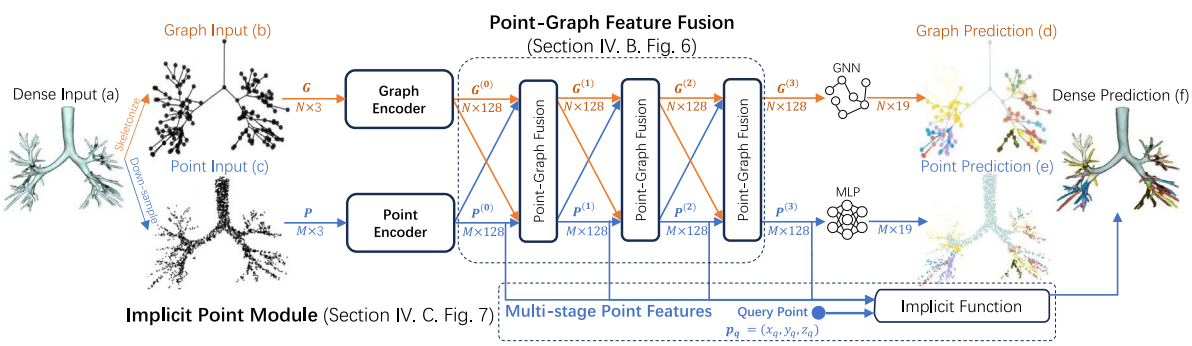

Fig. 6. Overview of the Proposed Implicit Point-Graph Network (IPGN) for Pulmonary Tree Labeling. The pipeline pre-processes dense volume to graph and point cloud input. The *Point-Graph Fusion* layers enhance point features with graph context with differentiable feature fusion learning, and the *Implicit Point Module* produces dense prediction based on the deep sparse representation efficiently.

We first introduce the general notation rule for both point and graph elements. While the coordinates of M points and N graph nodes are represented as $\mathbf{P}$ and $\mathbf{G}$, single point or graph element is expressed as $\mathbf{p}$ and $\mathbf{g}$, where $\mathbf{P} = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_m\}$, $\mathbf{G} = \{\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_n\}$. The superscript notation $\mathbf{p}^{(i)}$ represents an element's feature at the ith network layer.

At input, the 3-dimensional $\{x, y, z\}$ point coordinates, $\mathbf{P} \in \mathbb{R}^{M\times 3}$ and graph nodes, $\mathbf{G} \in \mathbb{R}^{N\times 3}$ are utilized as initial feature. We use a point neural network, and a graph neural network as initial feature encoders, from which we extract a 128-dimensional intermediate feature for each point and graph node, expressed as $\mathbf{P}^{(0)} \in \mathbb{R}^{M\times 128}$ and $\mathbf{G}^{(0)} \in \mathbb{R}^{N\times 128}$.

Subsequently, initial features from both branches $\mathbf{P}^{(0)}$ and $\mathbf{G}^{(0)}$ are incorporated within one or multiple *Point-Graph Fusion* layers, which allow for two-way feature integration based on feature propagation (Rossi et al., 2021) and ball-query&grouping (Qi et al., 2017). Let the input to a Point-Graph Fusion layer be defined as $\mathbf{P}^{(i-1)}$ and $\mathbf{G}^{(i-1)}$, the feature out of the fusion layer is $\mathbf{P}^{(i)}$ and $\mathbf{G}^{(i)}$. The last Point-Graph Fusion layer outputs $\mathbf{P}^{(l)}$ and $\mathbf{G}^{(l)}$ after l Point-Graph Fusion layers for deep feature fusion. Finally, a lightweight MLP network and a GNN projects the fusion feature to 19-dimensional vectors for graph (Fig. 6(d)) and point predictions (Fig. 6(e)).

An *Implicit Point Module* is further introduced to reconstruct the dense volumes, which consists of a feature propagation process and an MLP network. As features are extracted by the Point-Graph Network, the Implicit Point Module leverages the extracted multi-stage point features for fast dense volume segmentation. We define the points that are not necessarily in the original point cloud, but are in the foreground of the tree volume, thus requiring classification, as query points. Given a query point $\mathbf{p}_q$ with at arbitrary coordinates, the module locates $\mathbf{p}_q$'s k-nearest point elements from the original point cloud: $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_k\}$, and extracts their multi-stage features $\{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_k\}$ from the backbone Network for feature propagation into a multi-stage representation $\mathbf{z}_q$ of the query point $\mathbf{p}_q$. After propagating the point feature $\mathbf{z}_q$, the MLP network $\mathcal{H}$ is utilized to make class predictions. By applying this process to all foreground points, we can efficiently generate a dense volume reconstruction (Fig. 6(f)).

To avoid naming ambiguity, we refer to the aforementioned complete network as *Implicit Point-Graph Network (IPGN)*, and that sans the implicit part as the backbone network, *Point-Graph Network (PGN)*.

4.2. Point-graph feature fusion

The essence of our point-graph fusion learning approach lies in leveraging coordinate information as a basis for querying neighboring elements in the opposite branch. To achieve feature integration, We adopt ball-query & grouping for point-to-graph feature merging and feature propagation for graph-to-point feature merging.

For the ball-query & grouping method in the ith Point-Graph Fusion layer, a graph node $\mathbf{g}$ searches for all opposite point elements within a given ball with radius r as $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_b\}$. Then, an MLP module $\mathcal{F}_1 : \mathbb{R}^D \to \mathbb{R}^D$ independently project the point feature vectors to an updated representation of all queried points, Then, a feature-wise max-pooling

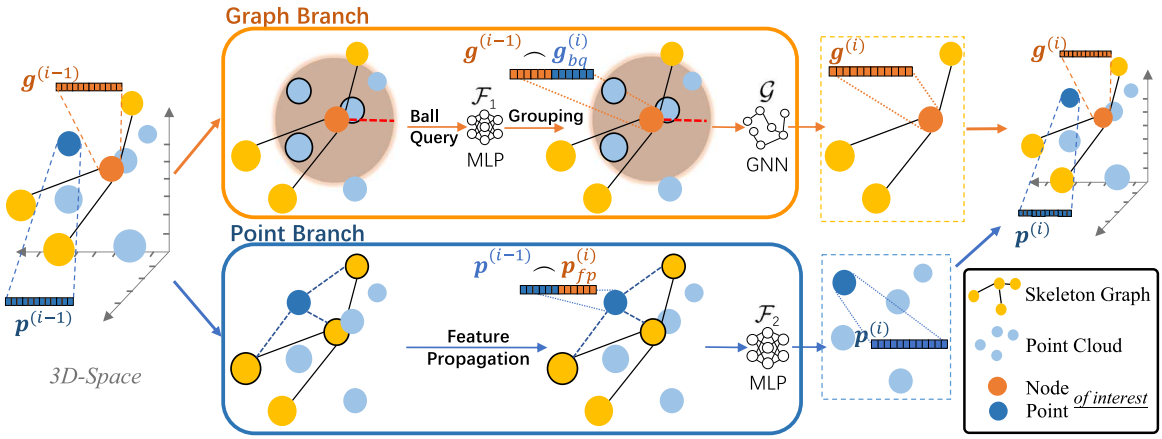

Fig. 7. The Detailed Operation of a Point-Graph Fusion layer. The figure only shows the operations for one node, and one point, which are highlighted as Node/Point *of interest*. In reality, this process is parallel for all nodes and points. Here, $\mathcal{F}_1$ and $\mathcal{F}_2$ are MLP networks while $\mathcal{G}$ represents a graph neural network.

layer aggregates all updated point features as point representation of the node $\mathbf{g}$, expressed as:

$$\mathbf{g}_{bq}^{(i)} = \max_j \left(\mathcal{F}_1(\mathbf{p}_j^{(i-1)})\right) \tag{1}$$

Subsequently, the ball-queried feature $\mathbf{g}_{bq}^{(i)}$ is combined with the current feature $\mathbf{g}^{(i-1)}$ before using a shallow graph attention network (Velickovic et al., 2017) $\mathcal{G} : \mathbb{R}^{2D} \rightarrow \mathbb{R}^{D_{next}}$ to perform graph convolution for feature fusion, resulting an updated feature representation of the node, $\mathbf{g}^{(i)} \in \mathbb{R}^{D_{next}}$, as input to the next Point-Graph Fusion layer.

For feature fusion from graph to point, feature propagation is utilized. In the process, each query point p with feature $\mathbf{p}^{(i-1)} \in \mathbb{R}^D$ at the ith fusion layer locates its k-nearest graph nodes $\{\mathbf{g}_1, \mathbf{g}_2, \ldots, \mathbf{g}_k\}$ in coordinate space. With k-nearest neighbors, the query point $\mathbf{p}$ acquires summarized graph feature $\mathbf{p}_{fp}^{(i)}$ by weighted summation (Eq. (2)) of the k node features $\{\mathbf{g}_1^{(i-1)}, \mathbf{g}_2^{(i-1)}, \ldots, \mathbf{g}_k^{(i-1)}\} \in \mathbb{R}^D$, where the weights are based on the normalized reciprocal distances. Let the distance between the query point and k neighbor nodes be $\{d_1, d_2, \ldots, d_k\}$, the propagation can be expressed as:

$$\mathbf{p}_{fp}^{(i)} = \frac{\sum_{j=1}^{k} \mathbf{g}_j^{(i-1)} \times \frac{1}{d_j}}{\sum_{l=1}^{k} \frac{1}{d_l}} \tag{2}$$

Then, the aggregated feature for point $\mathbf{p}$, $\mathbf{p}_{fp}^{(i)} \in \mathbb{R}^D$ is concatenated with the incoming point feature $\mathbf{p}^{(i-1)}$ to create $\mathbf{p}_{concat}^{(i)} \in \mathbb{R}^{2D}$. Finally, an MLP module $\mathcal{F}_2 : \mathbb{R}^{2D} \rightarrow \mathbb{R}^{D_{next}}$ projects the concatenated point feature to the input dimension of the next layer as $\mathbf{p}^{(i)} \in \mathbb{R}^{D_{next}}$.

4.3. Implicit dense volume reconstruction

To acquire dense volume segmentation results, the naive method is to sample all points from the pulmonary tree and group them into many non-overlapping original point clouds for multiple inferences. However, acquiring a dense volume prediction result from repeated point cloud prediction is computationally inefficient (example in Section 4.4.2) because the graph input remains identical and the point cloud is globally invariant across different inferences.

To improve computation efficiency for dense volume reconstruction, we propose the *Implicit Point Module* in Fig. 8 for efficient and arbitrary point inference. This module effectively interpolates the extracted original point cloud feature from the backbone Point-Graph Network as an feature field to enable fast dense volume reconstruction in a 3-step process.

First, for arbitrary query point coordinates $\mathbf{p}_q = (x_q, y_q, z_q) \in \mathbb{R}^3$ as input, its k nearest-neighbor points $\{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_k\}$ in the original point cloud (Fig. 8(a)) are located. Second, for each nearest neighbor point

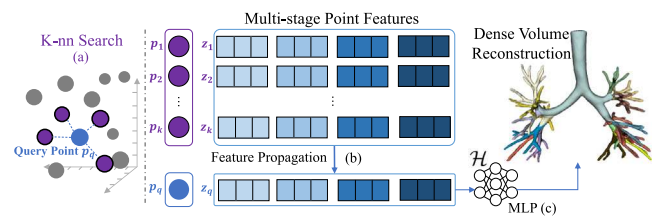

Fig. 8. Implicit Point Module. For any query point, the Implicit Point Module consumes multi-stage features from a Point-Graph Network with feature propagation and a neural network to provide a label.

$\mathbf{p}_k$, its corresponding features in different stages of the Point-Graph feature fusion are extracted and concatenated to form a multi-stage feature vector $\mathbf{z}_k = \{\mathbf{p}_k^{(0)} \frown \mathbf{p}_k^{(1)} \frown \ldots \frown \mathbf{p}_k^{(i)}\}$, where i denotes the number of Point-Graph Fusion layers. A feature propagation (Fig. 8(b)), similar to Eq. (2), is performed to aggregate $\{\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_k\}$ into the feature representation $\mathbf{z}_q$ for the query point. Finally, an MLP network $\mathcal{H}$ projects the feature $\mathbf{z}_q$ into a 19-dimensional vector for final classification (Fig. 8(c)). For dense volume segmentation results, we simply sample all foreground points and query through the module for prediction inference.

As the proposed PGN is regarded as the backbone point-based method, the Implicit Point Module is considered an additional accelerator for dense reconstruction, specializing in generating predictions for arbitrary location based on sparse features from the input point cloud. Therefore, from the perspective of the point-based method, the Implicit Point Module exists is an computation overhead at inference stage that sacrifice an insignificant amount of network (Fig. 8 $\mathcal{H}$) storage memory, in exchange for largely improved computation efficiency in dense reconstruction.

4.4. Model details

The IPGN is a customizable pipeline. The initial graph encoder utilizes 11 GAT layers and 128 channels. For ball query in point-to-graph fusion, we set ball radius $r = 0.1$, the maximum queried point to be 24, and employ a 3-layer GAT network to represent $\mathcal{G}$ in Fig. 7. For feature propagation in both graph-to-point fusion and the Implicit Point Module, we set k=3 for the k-nearest neighbor search. During the entire Point-Graph Network backbone, we set the intermediate fusion features to be 128-dimensional.

4.4.1. Training

At the point branch, we experimented with two point models, PointNet++ (Qi et al., 2017) or Point Transformer (Zhao et al., 2021) as initial feature encoders. For the graph encoder, we apply an 11-layer GAT (Velickovic et al., 2017) as the initial feature encoder. Prior to the training of the network, the feature encoders at point and graph branches are independently trained on the corresponding point and graph segmentation tasks and kept frozen.

The Point-Graph Network and the Implicit Point Module are trained simultaneously. First, we perform a forward pass on the Point-Graph Network, generating multi-stage point and graph features as well as predictions for the M ($6k$) input points and N graph elements. Subsequently, to train the Implicit Point Module, another set of M' ($6k$) foreground points are randomly sampled, and perform forward inference on the Implicit Point Module for M' predictions. After acquiring $M + M'$ point and N graph predictions, we apply the Cross-Entropy loss for training.

The two point encoder candidates: PointNet++ and Point Transformer, are both trained for 120 epochs with a learning rate of 0.002 while the GAT graph encoder is trained for 240 epochs with a learning rate of 0.02. The IPGN pipeline is trained for 100 epochs with a learning rate of 0.01 and for every 45 epochs, the learning rates are divided by 2. To avoid overfitting, we employ random rotation, shift, and scaling as data augmentation during training.

4.4.2. Inference

At inference time, the dense volume reconstruction procedure for IPGN involves a single point cloud inference through the backbone Point-Graph Network (PGN) and the inference of the remaining query points using the Implicit Point Module, as detailed in Section 4.3.

More specifically, to perform dense reconstruction with IPGN, we first execute pre-processing steps including skeletonization (Section 3.2.2) and down-sampling to acquire a graph and a point cloud. The graph and the original point cloud are then processed using the Point-Graph Network to obtain multi-stage point-graph features on the point cloud. With these extracted features, we gather all remaining query points belonging to the dense volume foreground and group them into multiple point cloud batches. For example, a dense volume of the airway tree contains approximately $180k$ foreground points, which we group into 4 batches with a batch size of 8 and each point cloud instance containing $6k$ points. Each batch is then iteratively fed into the Implicit Point Module as query points for highly efficient class predictions (Fig. 8).

Comparatively, for an arbitrary point-based backbone, such as the proposed Point-Graph Network or Point Transformer (Zhao et al., 2021), dense volume acquisition involves multiple full-model inferences on many point clouds that collectively assemble the dense volume. However, this method is computationally inefficient, as shown in Section 5.3 and Table 3.

5. Experiments

5.1. Experiment setting

By default, all experiments use dense volume as initial input. While CNN is natural for manipulating dense volume, we pre-process the dense volume to point clouds and skeleton graphs for point and graph experiments. We present the experiment metrics at the voxel-level, representing overall performance, and graph-level, representing performance at key locations in the structure. Although voxel and point-based methods do not inherently provide evaluations at the graph-level, we acquire the predictions at these key coordinates to provide the evaluation at key points for voxel/point methods. Moreover, voxel-level dense volume performance evaluation can be achieved using CNN methods, point-based methods with repeated inferences and graph-based models after post-processing (Section 5.1.3). Therefore, the evaluations in voxel-level, and graph-level across CNN, point, and graph methods are consistent and fair.

5.1.1. CNNs

In CNN experiments, given a 3D input, the task involves providing semantic segmentation prediction for the pulmonary tree in the image and all CNN methods apply a combination of dice loss and the cross-entropy loss for training. During testing, the image background is excluded from the metric computation.

Among the CNN experiments, we employ 3D-Unet (Çiçek et al., 2016) as the basic setup. Based on the 3D-Unet, we also implement a multi-task key-point regression method (Tan et al., 2021), abbreviated as "3D-Unet + KP". More specifically, an additional regression prediction head predicts a heatmap representing how likely is the location of a graph node as a key point. In these two CNN-based experiments, data are down-sampled to $96 \times 96 \times 96$ due to limited GPU memory during training and validation. During testing, the input is down-sampled for inference and re-scaled to the original dimension $N \times 512 \times 512$ for evaluation.

To address the issue of high memory usage and information loss by compromised resolution, we apply a sliding window approach in another CNN experiment, in which local 3D patches with dimension $96 \times 96 \times 96$ from the original image are used for training and validation purposes. During testing, the predictions obtained from the sliding-window technique were assembled back onto the original image for evaluation.

5.1.2. Point clouds

Point-based experiments involve treating a set of tubular voxels of a pulmonary tree as a point cloud for sparse representation for modeling. At the output, point-based model provides per-point classification prediction.

During training and validation of the experiments, we randomly sampled $6k$ foreground elements as point cloud input for this dataset, as sampling more than $6k$ points consumes more resources but does not increase performance. During testing, all foreground points are sampled, randomly permuted, and grouped into multiple point clouds. Then each point cloud containing $6k$ points is iteratively fed into the model for evaluation. Consequently, the inference processes provide a prediction for all foreground elements as dense volume results. PointNet (Qi et al., 2016), PointNet++ (Qi et al., 2017) and Point Transformer (Zhao et al., 2021) are tested as baselines.

5.1.3. Graphs

Graph experiments utilize the skeleton-graph generated from dense volume (Bumgarner and Nelson, 2022; Lee et al., 1994) as the graph structure, evaluating networks' ability to recognize key points and structural connections within the skeleton tree. These abilities are respectively represented by dice scores on graph nodes and edges.

In these experiments, we leverage multiple graph neural networks (GNNs) such as GAT (Velickovic et al., 2017), GraphSage (Hamilton et al., 2017), and a graph network with pre-trained point-based features as input. To identify an optimal configuration for graph-based networks, we conducted experiments using different GAT layers and feature embedding sizes (see Table 7) on the PTL dataset. We concluded that a configuration of 14 layers with 512-dimensional features is optimal, as it prevents under-fitting while maintaining high efficiency. Deeper GNNs fail to produce higher performance due to the known oversmoothing effect (Cai and Wang, 2020) in deep GNNs, and larger feature embeddings also did not yield better results. Consequently, we applied a 14-layer network for each graph baseline. After acquiring node features, features from the source node and destination node are interpolated by averaging for edge features before an MLP network projects all features to 19 dimensions for final classification.

Unlike voxel/point methods, Graph Neural Networks that perform learning on the skeleton graph provide evaluation at the graph level but not at the voxel level. To address this, we implemented a post-processing technique to dilate graph-based predictions to dense volume predictions. Specifically, for any given voxel, the algorithm assigns the label of the nearest graph element as its own label. Therefore, all graph baselines also provide voxel-level prediction metrics.

Table 2
Model performance in dice score at voxel and graph level on the PTL dataset. Baseline methods using different feature representations are presented to compare against the proposed methods on the Pulmonary Tree Labeling (PTL) dataset. PGN=Point-Graph Network without implicit fields; IPGN=Implicit Point Graph Network; KP=key point. *: Graph-level predictions from Voxel/Point Networks are acquired by inferencing on the node and edge locations.

Methods	Representation			Airway			Artery			Vein		
	Voxel	Point	Graph	Voxel (%)	Graph/KP* (%)		Voxel (%)	Graph/KP* (%)		Voxel (%)	Graph/KP* (%)	
					Node	Edge		Node	Edge		Node	Edge
Voxel/Point												
3D-Unet (down-sampled) (Çiçek et al., 2016)	✓			58.5	61.3	62.3	61.4	62.0	63.1	54.0	55.2	55.0
3D-Unet (down-sampled) + KP (Tan et al., 2021)	✓			56.9	62.2	61.9	61.3	62.1	63.1	55.0	55.4	54.8
3D-Unet (sliding-window) (Çiçek et al., 2016)	✓			39.8	40.3	41.2	22.5	23.5	24.1	28.7	29.3	28.7
PointNet (Qi et al., 2016)		✓		79.1	80.5	79.6	80.0	80.5	80.4	70.8	70.8	70.1
PointNet++ (Qi et al., 2017)		✓		82.9	84.0	83.5	82.5	82.8	83.5	75.7	75.7	75.6
Point Transformer (Zhao et al., 2021)		✓		87.8	87.2	87.4	86.5	87.2	87.4	77.7	77.2	78.0
Graph												
GCN (Kipf and Welling, 2016)			✓	82.8	85.5	83.0	79.6	82.1	80.4	73.7	73.5	70.4
GIN (Xu et al., 2018)			✓	84.6	88.8	85.6	81.4	84.9	82.8	74.0	74.3	70.9
GraphSage (Hamilton et al., 2017)			✓	86.0	91.7	88.0	82.4	86.7	84.5	75.9	79.1	75.0
HyperGraph (Yu et al., 2023)			✓	86.0	93.0	88.8	82.2	89.0	85.7	75.7	79.4	75.8
HyperGraph w/ Handcrafted Feat (Yu et al., 2023)			✓	85.8	92.8	88.7	81.8	88.6	85.2	75.7	79.5	75.4
GAT (Velickovic et al., 2017)			✓	86.3	92.7	89.1	83.4	88.7	86.3	76.0	79.0	75.3
GAT w/ Handcrafted Feat (Yu et al., 2023)			✓	86.3	92.5	88.8	83.7	88.4	86.2	76.1	78.1	74.1
Point + Graph												
GAT w/ PointNet++ Feat (Qi et al., 2017)		✓	✓	86.7	94.6	91.8	83.8	90.5	88.6	76.3	80.4	77.0
PGN (PointNet++ (Qi et al., 2017))		✓	✓	87.7	**95.1**	92.3	86.8	**92.9**	**89.9**	77.4	**82.8**	**79.4**
IPGN (PointNet++ (Qi et al., 2017))		✓	✓	87.5	**95.1**	92.3	86.6	**92.9**	**89.9**	77.5	**82.8**	**79.4**
PGN (Point Transformer (Zhao et al., 2021))		✓	✓	88.4	95.0	**92.7**	**87.2**	92.6	**89.9**	**78.6**	82.3	78.8
IPGN (Point Transformer (Zhao et al., 2021))		✓	✓	**88.5**	95.0	**92.7**	**87.2**	92.6	**89.9**	78.3	82.3	78.8

5.1.4. Ours

As discussed in depth in Section 4, we perform experiments combining point learning with graph learning. For the proposed Point-Graph network, the input and output setups for the point branch and graph branch are identical to those of point experiments and graph experiments. To speed up dense reconstruction, we incorporate the Implicit Point Module for point-based prediction, evaluated at voxel-level.

5.2. Model performance comparison

In this section, we perform a comparative analysis based on Table 2. The statistics presented in this section are the average performance over the 3 pulmonary structures. As previously discussed, we will treat graph elements as key locations in the tree structures. Therefore, "graph-level" is synonymous to "key point" in evaluation.

For voxel-based methods, the classic 3D-Unet (Çiçek et al., 2016) is used to implement multiple baselines. Two 3D-Unet (Çiçek et al., 2016; Tan et al., 2021) methods based on down-sampled input yield unsatisfying dice metrics at voxel-level, indicating that training CNN methods on reduced resolution leads to inferior modeling. As an attempt to train on the original resolution without memory restriction, the 3D-Unet (Çiçek et al., 2016) experiment with sliding-window strategy reports the poorest performances. Among point-based methods, the Point Transformer (Zhao et al., 2021) achieves the overall best voxel-level dice performances over PointNet++ (Qi et al., 2017) and PointNet (Qi et al., 2016), reaching an average of 84% in dice, which largely exceeds performances by any CNN methods. As for the graph-level results with voxel/point-based methods, namely the voxels/points performance at graph elements' locations, they are closely aligned with the respective voxel-level results, as there is no particular focus on key locations in these methods.

Among GNN methods with graph-only context, GAT (Velickovic et al., 2017) model, with minimal tuning, outperforms all previous baselines with a considerable margin, reaching 86.8% in dice score for node, displaying the advantage of tailored graph learning on the skeleton, focusing on key locations. In terms of graph-based voxel-level results derived from post-processing(Section 3.2.2), the GAT (Velickovic et al., 2017) model also achieves the best outcome, reaching 81.9% on average. Given that GNN baselines are highly light-weight and lack shape context, the reconstruction results are auspicious, indicating that the skeleton-graph already provides salient information.

In addition to graph modeling with 3D coordinate features, the performance of the GAT model, and a hypergraph (Yu et al., 2023) model with handcrafted features (Yu et al., 2023) are presented. The handcrafted feature for the graph nodes are carefully curated (Details in Section 5.5.1) to best resemble the original description (Yu et al., 2023), including structural, positional and morphological features. Compared to applying coordinate features as input, GAT with handcrafted features suffers from a slight drop in overall performance. Similarly, applying handcrafted features to the hypergraph (Feng et al., 2019) model also translates to a performance drop. These results suggest that in graph settings, tailor-designed features do not provide more valuable information than 3D coordinates. More insights with experiments are provided in Section 5.5.1.

For settings with both graph and point context, GAT with pre-trained PointNet++ feature has the lower performance compared to the proposed methods. Nevertheless, it still outperforms any baseline that does not involve point-graph fusion-feature by a considerable margin. Such a gap in performance indicates that the integration of point-context into graph learning is beneficial. For the proposed Point-Graph Network and IPGN, their performances beat all baselines in metrics at both levels, displaying superiority in point-graph fusion learning.

5.3. Dense volume reconstruction

In this subsection, we focus on the Implicit Point Module (Fig. 8), which aims to enhance the efficiency of labeled dense volume reconstruction (Fig. 6(f)). As discussed previously in Section 1, the integration of the implicit module is necessary because high-performing point models like Point Transformer (Zhao et al., 2021) could still be computationally expensive at test time: with repeated inferences for dense prediction, its computation cost grows in cubicle, similar to CNN. Experiments are completed to compare the efficiency using the proposed IPGN against only using the backbone Point-Graph Network as well as various baseline methods. The reconstruction efficiency is represented as the average run-time in seconds, for dense volume prediction across the PTL test set.

Regarding volume reconstruction efficiency, the test time of convolutional methods, graph-based method, point-based method, and our

Table 3
Inference speed and segmentation metrics for dense volume reconstruction. This table compares the dense volume segmentation test time and dice score across voxel, point, graph-based, and point-graph fusion methods. The test times are measured in seconds while dice score presents segmentation quality.

Methods	Model	Airway				Artery				Vein			
		Time (s)	Voxel (%)	Node (%)	Edge (%)	Time (s)	Voxel (%)	Node (%)	Edge (%)	Time (s)	Voxel (%)	Node (%)	Edge (%)
Voxel	3D-Unet (Çiçek et al., 2016) (downsampled)	8.16	58.5	61.3	62.3	8.02	61.4	62.0	63.1	7.46	54.0	55.2	55.0
Voxel	3D-Unet (Çiçek et al., 2016) (sliding-window)	9.43	39.8	40.3	41.2	11.88	22.5	23.5	24.1	12.44	28.7	29.3	28.7
Point	PointNet++ (Qi et al., 2017)	4.51	82.9	84.0	83.5	9.17s	82.5	82.8	83.5	9.65	75.7	75.7	75.6
Point	Point Transformer (Zhao et al., 2021)	10.97	87.8	87.2	87.4	21.94	86.5	87.2	87.4	23.25	77.7	77.2	78.0
Graph	GAT (Velickovic et al., 2017)	**1.15**	86.3	92.7	89.1	2.05	83.4	88.7	86.3	2.24	76.0	79.0	75.3
Point + Graph	PGN (PointNet++ (Qi et al., 2017))	5.65	87.7	95.1	92.3	12.40	86.8	**92.9**	**89.9**	13.16	77.4	**82.8**	**79.4**
Point + Graph	IPGN (PointNet++ (Qi et al., 2017))	1.30	87.5	95.1	92.3	**1.49**	86.6	**92.9**	**89.9**	**1.57**	77.5	**82.8**	**79.4**
Point + Graph	PGN (Point Transformer (Zhao et al., 2021))	12.10	88.4	**95.0**	**92.7**	24.28	**87.2**	92.6	**89.9**	25.81	**78.6**	82.3	78.8
Point + Graph	IPGN (Point Transformer (Zhao et al., 2021))	2.32	**88.5**	**95.0**	**92.7**	2.29	**87.2**	92.6	**89.9**	2.39	78.3	82.3	78.8

methods are documented. For all setups[1], we present the inference time and quality for dense volume segmentation in Table 3. Apart from model inference costs, test time is composed of various operations in different setups. For CNN with down-sampled input, test time includes down-sampling and up-sampling operations. For graph baselines, time measurement takes post-processing (Section 5.1.3) into account. For point-based networks, dense reconstruction time consists of multiple batched point cloud inferences. Finally, test time for IPGN includes a forward pass on the Point-Graph Network and the subsequent Implicit Point Module inference on all remaining points.

The results demonstrated the superiority of the IPGN in dense volume reconstruction. Qualitatively, the IPGN, with predictions from the Implicit Point Module rather than the backbone PGN, achieve nearly identical performance as the Point-Graph Network. In terms of efficiency, IPGN only requires 1.5 s on average with PointNet++ (Qi et al., 2017) and 2.3 s with Point Transformer (Zhao et al., 2021) as encoder. This reconstruction efficiency of IPGN is largely attributed to the Implicit Point Module. Let us explain the computational analysis of PGN and IPGN using dense reconstruction of the airway as an example. As shown in Table 3, the average inference time for PGN is 5.65 s, which can be broken down into an average of 3.95 batches per airway, with each batch containing 8 point clouds of $6k$ points, taking 1.43 s. In contrast, IPGN only requires running a single batch of $6k$ points through the PGN, taking 1.03 s, followed by dense volume reconstruction of all query points in 0.27 s, totaling just 1.30 s. Thus, by introducing the Implicit Point Module, we achieve a reduction of over 4 s with insignificant computational overhead. For more complex cases, such as inputs with more points (arteries and veins) and computationally intensive backbones (Point Transformer), the time savings with IPGN are even more significant.

Although the graph-based method records a similar time cost, it fails to generate quality results. Further, all other methods are relatively inefficient. Therefore, the proposed IPGN pipeline is the all-around optimal solution for dense volume segmentation for pulmonary structures.

[1] To make it easily reproducible, all the speed measurements were conducted on a free Google Colab T4 instance—CPU: Intel(R) Xeon(R) CPU @ 2.00 GHz, GPU: NVIDIA Tesla T4 16Gb, memory: 12Gb.

These findings highlight the practical utility of the proposed module, as it allows for fast and efficient point inference without compromising quality. The Implicit Point Module serves as a valuable contribution in overcoming the computational challenges associated with analyzing the pulmonary tree, enabling rapid decision-making in clinical settings.

5.4. Qualitative analysis with visualization

In the qualitative analysis section, we employ 2 concrete cases each from the airway, artery, and vein tree to demonstrate the efficacy of our method. Fig. 9(a–f) showcases the results of full pulmonary tree segmentation using graph-only method, point-only method, and the Implicit Graph-Point Network, against the ground truth image.

The graph-based predictions from Fig. 9(b-d) reveal instances of incorrect graph predictions leading to anomalous outcomes, where multiple class labels appear in the same branch, thereby inducing inconsistent branch predictions. For point-based method, due to its sparse representation in individual forward passes, point predictions often lose details and disrupt the borders between branches (Fig. 9(a, c, e)), resulting in volumes that lack smoothness and quality.

Conversely, the proposed IPGN offers various advantages. Firstly, our method accurately classifies distal branches, illustrated by dense volume prediction in Fig. 9 (d,f). Additionally, our method effectively defines clear and uninterrupted borders between the child branches at bifurcation points. This is especially valuable when a parent node branches into multiple child branches belonging to different classes, as demonstrated in Fig. 9(b,e). By integrating graph and point context into the modeling process, our method enhances the segmentation capabilities at bifurcation points and distal branches, ultimately producing smoother and more uniform branch predictions.

5.5. Ablation experiments

5.5.1. Feature for graph modeling

In this section, we examine the effect of different input features for the skeleton graph node. The candidates are 3D coordinates feature and handcrafted feature. As coordinate feature is simply the $\{x, y, z\}$ coordinates, handcrafted feature follows a similar feature design from Yu et al. (2023).

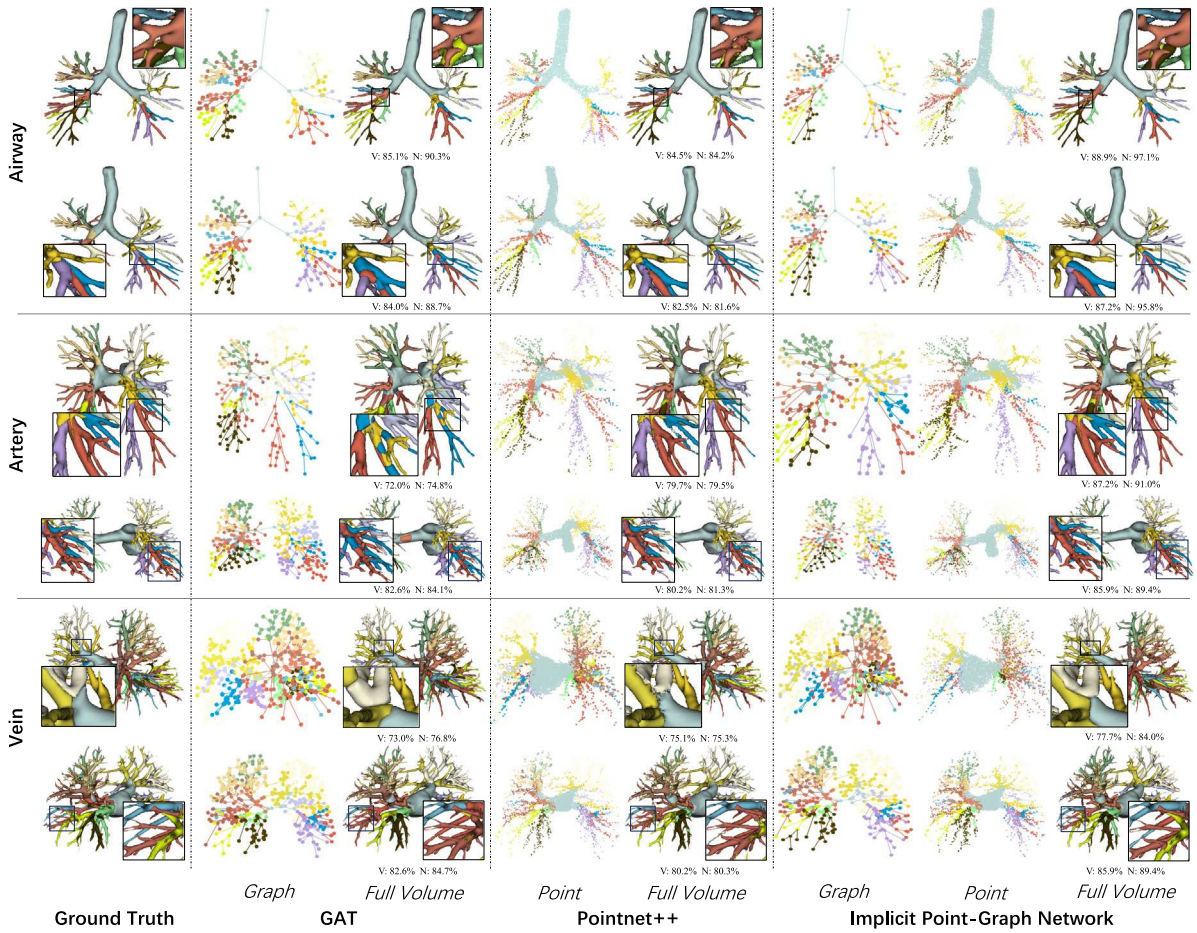

Fig. 9. Visualization of Segmentation Results. This figure displays segmentation results using GAT (Velickovic et al., 2017) w/ post-processing (Section 5.1.3), PointNet++ (Qi et al., 2017) and IPGN (with GAT and PointNet++ backbones) methods along with the ground truth. The initial-form predictions (graph or point cloud) before dense volume prediction are also presented. Below each dense volume reconstruction, we show voxel (V) dice and node (N) dice results.

In the experiment setup of Yu et al. (2023), a graph node represents an airway tree branch in volumetric space, with features including structural, positional, and morphological attributes derived from the tree branch. In our proposed pipeline, the volume is abstracted as a skeleton graph, with graph nodes representing bifurcation points or points along the centerline of branches. We slightly modified the original feature design to fit our skeleton graph setup, described as follows. Our version of the feature design can be adapted to airway, artery, and vein trees, instead of being limited to airway trees.

For structural features such as generation, number of children and siblings, these can be derived from the skeleton graph by treating the extra-pulmonary structure (class=19) as a starting point (first generation) and iteratively calculating the structural information for each branching point until reaching leaf nodes.

For positional features, the feature design indicates that the spatial locations of branches can be used in various ways, such as using an anchor location to establish coordinate systems. All positional features can be acquired for the graph because each node is represented by its 3D coordinates. For the airway tree, we follow the original design by treating the end of the trachea as the anchor. For the artery and vein trees, we use the average coordinates of all nodes representing extra-pulmonary structures as anchors. Positional features also include creating a bounding box for tree structures in the left and right lung to establish another coordinate system. Additionally, branches are sorted, and their sorted orders and angles are also used as positional features. In our implementation, rather than each branch, we rank each graph node in the three spatial dimensions and calculate the angles of edges that connect each node.

For morphological features, we use the length of branches and the projected lengths of branches in three axes as features. This can be implemented at the node level by calculating the corresponding total lengths of the edges that connect to a node.

In our experiments, we use a GAT (Velickovic et al., 2017) and a 5-layer MLP network to evaluate the impact of coordinate features and handcrafted features, reported in Table 4. Based on the MLP experiment, handcrafted feature achieves 78.4% dice results on average while raw coordinate only reports 71.5%. Conversely, in the experiment with GAT (Velickovic et al., 2017), applying the coordinate feature achieves overall best performances, reaching 86.8% while the handcrafted feature trails behind.

The results indicate that the handcrafted feature provides more information and produces better performance in an MLP setting, where the neural network simply learns a non-linear projection without any global or tree topology context. Once the connections between graph nodes are established through edge in graph modeling, the handcrafted feature completely loses its advantages over the raw coordinate feature, which implies that the handcrafted feature could be learned during graph learning and the learned graph-based feature is more beneficial to graph segmentation quality.

5.5.2. *Input selection for implicit point module*

In this section, we present the results of an ablation study where we explore the impact of multi-stage features using different layers of the intermediate feature vector as input to the implicit module.

In the experiment, we initially utilized the feature output of the final Point-Graph Fusion layer as the sole input to the implicit module. Motivated by the design of DenseNet (Huang et al., 2016), we

Table 4
Impact of input for graph modeling. We show the results using MLP and GAT (Velickovic et al., 2017) backbones on different feature inputs.

Method	Features		Airway		Artery		Vein	
	Handcrafted	Raw Coord	Node	Edge	Node	Edge	Node	Edge
MLP	✓		**80.8**	**81.2**	78.1	**77.6**	74.2	74.3
		✓	69.2	69.7	72.4	72.2	71.9	72.0
	✓	✓	80.7	80.9	**78.2**	**77.6**	**75.5**	**75.3**
GAT (Velickovic et al., 2017)	✓		92.0	89.5	88.0	85.6	77.8	73.4
		✓	**92.7**	89.1	**88.7**	**86.3**	**79.0**	**75.3**
	✓	✓	92.2	**89.2**	88.2	86.0	78.5	75.1

Table 5
Multi-stage point feature input for implicit point module. This table presents the dice results using different combinations of concatenated multi-stage features from the backbone network as input to the Point Implicit Module. (Point Encoder: Point Transformer (Zhao et al., 2021); Graph Encoder: GAT (Velickovic et al., 2017)).

	$\mathbf{P}^{(3)}$	$\mathbf{P}^{(2)} \frown \mathbf{P}^{(3)}$	$\mathbf{P}^{(1)} \frown \mathbf{P}^{(2)} \frown \mathbf{P}^{(3)}$	$\mathbf{P}^{(0)} \frown \mathbf{P}^{(1)} \frown \mathbf{P}^{(2)} \frown \mathbf{P}^{(3)}$
Airway	88.1	88.2	88.4	**88.5**
Artery	86.7	87.0	**87.0**	**87.2**
Vein	77.9	77.9	**78.1**	**78.3**

Table 6
Model performance on the original/corrupted data. We present experiment results on the original and corrupted version (Weng et al., 2023) of the PTL dataset for Point Transformer (Zhao et al., 2021) and the proposed IPGN with voxel-level dice score (%) as metric.

Methods	Airway	Artery	Vein	Average Δ
Point Transformer (Zhao et al., 2021)	87.8/85.4	86.5/85.0	77.7/74.1	2.5↓
IPGN (Point Transformer (Zhao et al., 2021))	88.5/88.0	87.2/86.1	78.3/76.2	1.2↓

experimented with adding feature outputs from shallower intermediate blocks, forming multi-stage features, and report the performance with each feature addition until the initial point feature.

Table 5 demonstrates that combining multi-stage features, along with the initial point (PointNet++ (Qi et al., 2017)) feature, enhances the predictive capabilities of our model, contributing to better performance. Finally, the best-performing configuration we observed involved using all available features, yielding results on par with the full modeling approach.

5.6. Robustness test on corrupted data

As the proposed pipeline is intended to be deployed in diverse environments, high-quality pre-extractions (binary segmentation) of the 3D tree structures from CT scans could be challenging due to the complex tubular shapes, especially at distal endpoints, where thin tubular branches in the resulting volume could be disconnected. However, we believe pulmonary tree segmentation and pulmonary tree labeling are distinct yet important issues, as standard segmentation methods might struggle with pulmonary tree labeling, as shown in Table 2 and Table 3 with the 3D Unet results.

To test the robustness of our model under poorly constructed trees, we performed additional experiments on a corrupted version of the PTL dataset, which contains pulmonary trees with disconnections (Weng et al., 2023). The results, presented in Table 6 of the manuscript, indicate that while both Point Transformer and IPGN are robust to volume-level corruption, the proposed method is relatively more robust compared to point-based methods. Overall, the corruption in branches does not pose a major threat to the overall segmentation of the target structures.

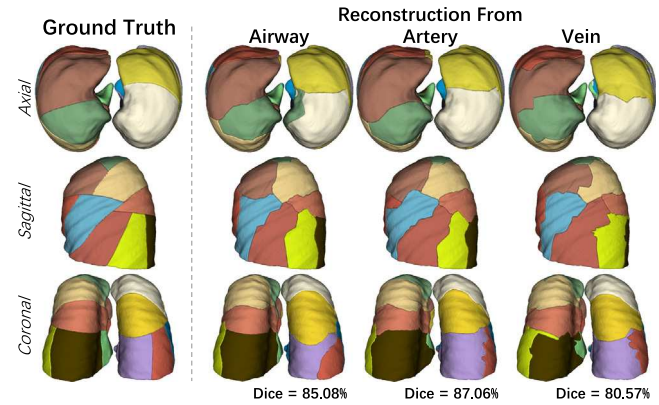

Fig. 10. Pulmonary Segments Visualization. A sample reconstruction of pulmonary segments based on the airway, artery, and vein is presented against ground truth in axial, sagittal and coronal views.

5.7. Model complexity

To demonstrate that the performance improvement of PGN/IPGN is not merely due to an increase in parameter count, we scaled up the size of both graph-based and point-based models to incorporate model complexity into the comparison. We recorded the model parameter count for the graph-based methods, point-based methods, and the proposed methods in Table 7.

For the graph-based models, we evaluated the highest-performing baseline, GAT (Velickovic et al., 2017), under different configurations. As the GAT layer grows from 11 to 25 layers and the maximum feature embedding grows from 256 to 1024, the parameter count increases considerably from $1.8M$ to $15.5M$, largely exceeding our methods. However, increasing the parameter count of the GAT model does not contribute to significant performance gains, which aligns with the known over-smoothing effect in deep graph modeling (Cai and Wang, 2020). Similarly for point-based methodsy, while increasing the model size may improve performance, it is far less effective than the enhancements provided by Point-Graph Fusion.

Our PGN/IPGN combines the advantages of point representation and graph representation. For example, among the $3.6M$ parameters of the PGN (PointNet++), $1.0M$ parameters are from PointNet++, $1.3M$ parameters are from GAT networks, and 1.3M parameters are from Point-Graph feature fusion layers (Section 4.2). As demonstrated, the performance improvement from this combination is highly rewarding.

Finally, while IPGN adds a small number of parameters ($0.1M$) compared to PGN, it results in a significant efficiency improvement (Table 3).

6. Extended application: Reconstruction of pulmonary segments without retraining

Pulmonary segments are subdivisions of the pulmonary lobes and are crucial for guiding segmentectomy, a widely applied surgical treatment for lung cancer with a higher survival rate than lobectomy (Saji et al., 2022). The definition of pulmonary segments is anatomically related to the pulmonary trees, rather than individual pixel texture features. On CT scans, while pulmonary lobes have visible boundaries, pulmonary segments do not. As illustrated in Fig. 3, the spatial location of pulmonary trees implicitly defines the boundaries of pulmonary segments. A pulmonary segment must spatially contain the pulmonary trees of the same class, with boundaries between neighboring segments established along the inter-segmental veins (Kuang et al., 2022). Therefore, the segmentation of pulmonary trees is an important prerequisite for reconstructing pulmonary segments, not the other way around.

Table 7
Model complexity. This table highlights the parameter counts for key competitive methods. The performance is represented as voxel-level dice (%).

Methods	Configuration		Airway	Artery	Vein	Parameters
	Feature	Layer				
PointNet++ (Qi et al., 2017)	512	4	82.9	82.5	75.7	1.0M
	1024	6	83.4	82.8	76.8	3.9M
Point Transformer (Zhao et al., 2021)	512	4	87.8	86.5	77.7	4.5M
	1024	6	88.1	86.7	78.0	12.5M
GAT (Velickovic et al., 2017)	128	11	86.0	81.4	74.0	1.3M
	256	11	86.2	81.8	74.4	1.8M
		14	86.2	82.9	75.4	2.2M
	512	11	86.0	82.5	74.8	3.9M
		14	86.3	83.4	76.0	5.5M
		20	87.0	84.2	76.1	6.1M
		25	86.6	83.2	75.7	8.6M
	1024	14	87.1	82.9	76.3	12.3M
		20	86.9	82.0	76.0	13.7M
		25	85.5	83.0	75.3	15.5M
PGN (PointNet++ (Qi et al., 2017))	*N/A*		87.7	86.8	77.4	3.6M
IPGN (PointNet++ (Qi et al., 2017))			87.5	86.6	77.5	3.7M
PGN (Point Transformer (Zhao et al., 2021))			88.4	**87.2**	**78.6**	7.2M
IPGN (Point Transformer (Zhao et al., 2021))			**88.5**	**87.2**	78.3	7.3M

In the proposed architecture, the Implicit Point Module plays a vital role in defining implicit boundaries between different classes within the pulmonary tree, allowing for efficient dense reconstruction of the pulmonary tree. As the module learns a point-graph feature field for labeling, the input coordinates are not constrained to the tubular voxel/point in our target structures. Consequently, any point within the pulmonary lobe can be assigned a class label. By utilizing the extracted point-graph feature field from the airway, artery, or vein tree, our module can accurately infer class information for all points within the lobe, following a similar procedure described in Section 5.3.

Precisely, for each subject, we first acquire the point-based feature field from the proposed Point-Graph Network. As the binary mask for pulmonary lobes (Fig. 1(a)) is given by the PTL dataset, we sample all foreground voxel/points in the mask, and iteratively perform query inferences on the Implicit Point Module based on the previously extracted feature field. This enables us to achieve a natural semantic reconstruction of the pulmonary segments, depicted in Fig. 10.

Compared to the ground truth, the tree-segmentation-induced pulmonary segments reconstruction achieves 79.89%, 81.85% and 76.16% as micro-average dice scores, respectively for pulmonary airway, artery, and vein. Future works can potentially produce better pulmonary segment reconstruction by integrating features from the three pulmonary trees or leveraging the explicit pulmonary lobe boundaries as additional guidelines.

7. Conclusion

In conclusion, we take an experimentally comprehensive deep-dive into pulmonary tree segmentation based on the compiled PTL dataset. A novel architecture Implicit Point-Graph Network (IPGN) is presented for accurate and efficient pulmonary tree segmentation. Our method leverages a dual-branch point-graph fusion model to effectively capture the complex branching structure of the respiratory system. Extensive experiment results demonstrate that by implicit modeling on point-graph features, the proposed model achieves state-of-the-art segmentation quality with minimum computation cost for practical dense volume reconstruction. The advancements made in this study could potentially enhance the diagnosis, management, and treatment of pulmonary diseases, ultimately improving patient outcomes in this critical area of healthcare.

CRediT authorship contribution statement

Kangxian Xie: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis. **Jiancheng Yang:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Donglai Wei:** Writing – review & editing, Writing – original draft, Validation, Investigation. **Ziqiao Weng:** Writing – review & editing, Writing – original draft, Visualization. **Pascal Fua:** Writing – review & editing, Writing – original draft, Supervision, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Jiancheng Yang reports a relationship with Dianei Technology that includes: equity or stocks. Other authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The Pulmonary Tree Labeling (PTL) dataset and associated code can be accessed at https://github.com/M3DV/pulmonary-tree-labeling.

Acknowledgments

This work was supported in part by the Swiss National Science Foundation.

References

Bumgarner, J.R., Nelson, R.J., 2022. Open-source analysis and visualization of segmented vasculature datasets with VesselVio. Cell Rep. Methods 2 (4), 100189.
Cai, C., Wang, Y., 2020. A note on over-smoothing for graph neural networks. arXiv arXiv:2006.13318.
Charbonnier, J.P., Pompe, E., Moore, C.M., Humphries, S.M., van Ginneken, B., Make, B.J., Regan, E.A., Crapo, J.D., van Rikxoort, E.M., Lynch, D.A., 2019. Airway wall thickening on CT: Relation to smoking status and severity of COPD. Respir. Med. 146, 36–41.
Chen, Z., Zhang, H., 2018. Learning implicit fields for generative shape modeling. In: Conference on Computer Vision and Pattern Recognition. pp. 5932–5941.

Chibane, J., Alldieck, T., Pons-Moll, G., 2020. Implicit functions in feature space for 3D shape reconstruction and completion. In: Conference on Computer Vision and Pattern Recognition. pp. 6968–6979.

Çiçek, Ö., Abdulkadir, A., Lienkamp, S.S., Brox, T., Ronneberger, O., 2016. 3D U-net: Learning dense volumetric segmentation from sparse annotation. In: Conference on Medical Image Computing and Computer Assisted Intervention.

Decramer, M., Rennard, S.I., Troosters, T., Mapel, D.W., Giardino, N.D., Mannino, D.M., Wouters, E.F.M., Sethi, S., Cooper, C.B., 2008. COPD as a lung disease with systemic consequences – clinical impact, mechanisms, and potential for early intervention. COPD: J. Chronic Obstr. Pulm. Dis. 5, 235–256.

Feng, Y., You, H., Zhang, Z., Ji, R., Gao, Y., 2019. Hypergraph neural networks. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, No. 01. pp. 3558–3565.

Gerard, S.E., Patton, T.J., Christensen, G., Bayouth, J.E., Reinhardt, J.M., 2019. FissureNet: A deep learning approach for pulmonary fissure detection in CT images. IEEE Trans. Med. Imaging 38, 156–166.

Gerard, S.E., Reinhardt, J.M., 2019. Pulmonary lobe segmentation using a sequence of convolutional neural networks for marginal learning. In: International Symposium on Biomedical Imaging. pp. 1207–1211.

Hamilton, W.L., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems.

Huang, G., Liu, Z., Weinberger, K.Q., 2016. Densely connected convolutional networks. In: Conference on Computer Vision and Pattern Recognition. pp. 2261–2269.

Huang, X., Yang, J., Wang, Y., Chen, Z., Li, L., Li, T., Ni, B., Zhang, W., 2022. Representation-agnostic shape fields. In: International Conference on Learning Representations.

Jin, L., Gu, S., Wei, D., Adhinarta, J.K., Kuang, K., Zhang, Y.J., Pfister, H., Ni, B., Yang, J., Li, M., 2023. RibSeg v2: A large-scale benchmark for rib labeling and anatomical centerline extraction. IEEE Trans. Med. Imaging.

Jumper, J.M., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Zídek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S.A.A., Ballard, A., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., Back, T., Petersen, S., Reiman, D.A., Clancy, E., Zielinski, M., Steinegger, M., Pacholska, M., Berghammer, T., Bodenstein, S., Silver, D., Vinyals, O., Senior, A.W., Kavukcuoglu, K., Kohli, P., Hassabis, D., 2021. Highly accurate protein structure prediction with AlphaFold. Nature 596, 583–589.

Kipf, T., Welling, M., 2016. Semi-supervised classification with graph convolutional networks. arXiv Preprint arXiv:1609.02907.

Kirby, M., Tanabe, N., Vasilescu, D.M., Cooper, J.D., McDonough, J.E., Verleden, S.E., Vanaudenaerde, B.M., Sin, D.D., Tan, W.C., Coxson, H.O., Hogg, J.C., 2019. Computed tomography total airway count is associated with number of micro-CT terminal bronchioles. Am. J. Respir. Crit. Care Med..

Kuang, K., Zhang, L., Li, J., Li, H., Chen, J., Du, B., Yang, J., 2022. What makes for automatic reconstruction of pulmonary segments. arXiv Preprint arXiv:2207.03078,

Lee, T., Kashyap, R., Chu, C., 1994. Building skeleton models via 3-D medial surface axis thinning algorithms. CVGIP, Graph. Models Image Process. 56 (6), 462–478.

Li, Y., Bu, R., Sun, M., Wu, W., Di, X., Chen, B., 2018. PointCNN: Convolution on X-Transformed points. In: Conference on Computer Vision and Pattern Recognition.

Lin, Z.H., Huang, S.Y., Wang, Y., 2020. Convolution in the cloud: Learning deformable kernels in 3D graph convolution networks for point cloud analysis. In: Conference on Computer Vision and Pattern Recognition. pp. 1797–1806.

Liu, Z., Tang, H., Lin, Y., Han, S., 2019. Point-voxel CNN for efficient 3D deep learning. arXiv arXiv:1907.03739.

Loud, P.A., Katz, D.S., Bruce, D.A., Klippenstein, D.L., Grossman, Z.D., 2001. Deep venous thrombosis with suspected pulmonary embolism: Detection with combined CT venography and pulmonary angiography. Radiology 219 (2), 498–502, PMID: 11323478.

Marcus, P.M., Bergstralh, E.J., Fagerstrom, R.M., Williams, D.E., Fontana, R.S., Taylor, W.R., Prorok, P.C., 2000. Lung cancer mortality in the Mayo Lung Project: impact of extended follow-up. J. Natl. Cancer Inst. 92 16, 1308–1316.

Mescheder, L.M., Oechsle, M., Niemeyer, M., Nowozin, S., Geiger, A., 2018. Occupancy networks: Learning 3D reconstruction in function space. In: Conference on Computer Vision and Pattern Recognition. pp. 4455–4465.

Nardelli, P., Jimenez-Carretero, D., Bermejo-Peláez, D., Washko, G.R., Rahaghi, F.N., Ledesma-Carbayo, M.J., Estépar, R.S.J., 2018. Pulmonary artery–Vein classification in CT images using deep learning. IEEE Trans. Med. Imaging 37, 2428–2440.

Nunes, C., Pereira, A.M., Morais-Almeida, M., 2017. Asthma costs and social impact. Asthma Res. Pract. 3.

Oizumi, H., Kato, H., Endoh, M., Inoue, T., Watarai, H., Sadahiro, M., 2014. Techniques to define segmental anatomy during segmentectomy. Anna. Cardiothoracic Surg. 3 2, 170–175.

Park, J.J., Florence, P.R., Straub, J., Newcombe, R.A., Lovegrove, S., 2019. DeepSDF: Learning continuous signed distance functions for shape representation. In: Conference on Computer Vision and Pattern Recognition. pp. 165–174.

Peng, S., Niemeyer, M., Mescheder, L.M., Pollefeys, M., Geiger, A., 2020. Convolutional occupancy networks. arXiv Preprint arXiv:2003.04618.

Qi, C., Su, H., Mo, K., Guibas, L.J., 2016. PointNet: Deep learning on point sets for 3D classification and segmentation. In: Conference on Computer Vision and Pattern Recognition. pp. 77–85.

Qi, C., Yi, L., Su, H., Guibas, L.J., 2017. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In: Advances in Neural Information Processing Systems.

Qin, Y., Chen, M., Zheng, H., Gu, Y., Shen, M., Yang, J., Huang, X., Zhu, Y.M., Yang, G.Z., 2019. AirwayNet: A voxel-connectivity aware approach for accurate airway segmentation using convolutional neural networks. arXiv arXiv:1907.06852.

Qin, Y., Zheng, H., Gu, Y., Huang, X., Yang, J., Wang, L., Yao, F., Zhu, Y.M., Yang, G.Z., 2020. Learning tubule-sensitive CNNs for pulmonary airway and artery-vein segmentation in CT. IEEE Trans. Med. Imaging 40, 1603–1617.

Quaderi, S., Hurst, J.R., 2018. The unmet global burden of COPD. Global Health Epidemiol. Genom. 3.

Rossi, E., Kenlay, H., Gorinova, M.I., Chamberlain, B.P., Dong, X., Bronstein, M.M., 2021. On the unreasonable effectiveness of feature propagation in learning on graphs with missing node features. arXiv arXiv:2111.12128.

Saji, H., Okada, M., Tsuboi, M., Nakajima, R., Suzuki, K., Aokage, K., Aoki, T., Okami, J., Yoshino, I., Ito, H., et al., 2022. Segmentectomy versus lobectomy in small-sized peripheral non-small-cell lung cancer (JCOG0802/WJOG4607l): a multicentre, open-label, phase 3, randomised, controlled, non-inferiority trial. Lancet 399 (10335), 1607–1617.

Shaw, R.J., Djukanović, R., Tashkin, D.P., Millar, A.B., du Bois, R.M., Corris, P.A., 2002. The role of small airways in lung disease. Respir. Med. 96 2, 67–80.

Shen, Y., Wan, C., Tian, P., Wu, Y., ou Li, X., Yang, T., An, J., Wang, T., Chen, L., Wen, F., 2014. CT-base pulmonary artery measurement in the detection of pulmonary hypertension: a meta-analysis and systematic review. Medicine 93 27, e256.

Smith, B.M., Hoffman, E.A., Rabinowitz, D., Bleecker, E., Christenson, S., Couper, D., Donohue, K.M., Han, M.K., Hansel, N.N., Kanner, R.E., Kleerup, E., Rennard, S., Barr, R.G., 2014. Comparison of spatially matched airways reveals thinner airway walls in copd. The multi-ethnic study of atherosclerosis (MESA) COPD study and the subpopulations and intermediate outcomes in COPD study (SPIROMICS). Thorax 69 (11), 987–996.

Tan, Z., Feng, J., Zhou, J., 2021. Sgnet: Structure-aware graph-based network for airway semantic segmentation. In: Conference on Medical Image Computing and Computer Assisted Intervention. Springer International Publishing, Cham, pp. 153–163.

Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio', P., Bengio, Y., 2017. Graph attention networks. arXiv Preprint arXiv:1710.10903.

Weng, Z., Yang, J., Liu, D., Cai, W., 2023. Topology repairing of disconnected pulmonary airways and vessels: Baselines and a dataset. arXiv:2306.07089.

Wickramasinghe, U., Jensen, P., Shah, M., Yang, J., Fua, P., 2022. Weakly supervised volumetric image segmentation with deformed templates. In: Conference on Medical Image Computing and Computer Assisted Intervention. Springer, pp. 422–432.

Wickramasinghe, U., Remelli, E., Knott, G., Fua, P., 2020. Voxel2mesh: 3d mesh model generation from volumetric data. In: Conference on Medical Image Computing and Computer Assisted Intervention. Springer, pp. 299–308.

Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., Xiao, J., 2015. 3D shapenets: A deep representation for volumetric shapes. In: Conference on Computer Vision and Pattern Recognition. pp. 1912–1920.

Xie, W., Jacobs, C., Charbonnier, J.P., van Ginneken, B., 2022. Structure and position-aware graph neural network for airway labeling. arXiv Preprint arXiv:2201.04532.

Xu, K., Hu, W., Leskovec, J., Jegelka, S., 2018. How powerful are graph neural networks? arXiv Preprint arXiv:1810.00826.

Yang, J., Gu, S., Wei, D., Pfister, H., Ni, B., 2021. RibSeg dataset and strong point cloud baselines for rib segmentation from CT scans. In: Conference on Medical Image Computing and Computer Assisted Intervention. Springer International Publishing, pp. 611–621.

Yang, J., Wickramasinghe, U., et al., 2022. ImplicitAtlas: learning deformable shape templates in medical imaging. In: Conference on Computer Vision and Pattern Recognition. pp. 15861–15871.

Yang, J., Zhang, Q., Ni, B., Li, L., Liu, J., Zhou, M., Tian, Q., 2019. Modeling point clouds with self-attention and gumbel subset sampling. In: Conference on Computer Vision and Pattern Recognition. pp. 3323–3332.

Yu, W., Zheng, H., Gu, Y., Xie, F., Yang, J., Sun, J., Yang, G.-Z., 2023. TNN: Tree neural network for airway anatomical labeling. IEEE Trans. Med. Imaging 42 (1), 103–118.

Zhang, M., Wu, Y., et al., 2023. Multi-site, multi-domain airway tree modeling. Med. Image Anal. 102957.

Zhao, H., Jiang, L., Jia, J., Torr, P.H., Koltun, V., 2021. Point transformer. In: International Conference on Computer Vision. pp. 16259–16268.

Zhou, Z., Siddiquee, M.M.R., Tajbakhsh, N., Liang, J., 2019. Unet++: Redesigning skip connections to exploit multiscale features in image segmentation. IEEE Trans. Med. Imaging 39 (6), 1856–1867.

Zhou, Z., Siddiquee, M.M.R., et al., 2018. Unet++: A nested U-net architecture for medical image segmentation. In: DLMIA MICCAI Workshop Vol. 11045. pp. 3–11.